



Universidad Veracruzana

Facultad de Estadística e Informática

Región Xalapa

Licenciatura en Ingeniería de Software

Desarrollo de un catálogo de artefactos de diseño orientados a la seguridad

Protocolo de investigación para acreditar Proyecto Guiado
Modalidad: Tesina

Presenta:

Carim Velázquez Chicuellar

Director:

Dr. Héctor Xavier Limón Riaño

Codirector:

Dr. Jorge Octavio Ocharán Hernández

Febrero 2026

“Lis de Veracruz: Arte, Ciencia, Luz”



Índice general

Índice general	1
Índice de figuras	2
Índice de tablas	2
I. Antecedentes	3
2. Planteamiento del problema	5
3. Objetivos	6
3.1. Objetivo general.....	6
3.2. Objetivos específicos.....	6
4. Justificación	7
5. Supuesto.....	8
6. Alcances y limitaciones.....	8
6.1. Alcances.....	9
6.2. Limitaciones	9
7. Marco teórico.....	9
7.1. Ingeniería de Software	9
7.2. Diseño de software	10
7.2.1. Modelo y artefacto.....	10
7.2.2. Atributo de calidad.....	10
7.3. Ciberseguridad y desarrollo seguro	11
7.3.1. Diseño de software seguro.....	11
7.3.2. Modelado de amenazas	12
7.3.3. Ataque	12
7.3.4. Vulnerabilidad	12
7.3.5. Amenaza.....	12
8. Método.....	13
9. Contenido del trabajo de investigación	15
10. Cronograma de actividades.....	17
Referencias.....	18

Índice de figuras

Figura I.I. Representación del método del trabajo de investigación. . ¡Error! Marcador no definido.

Índice de tablas

Tabla I.I. Cronograma de actividades..... 17

I. Antecedentes

Cuando se habla de Ingeniería de Software y de todo lo que conlleva, tarde o temprano se abordan invariablemente las fases que esta disciplina posee, mediante las cuales se sigue un proceso de desarrollo sistematizado. De acuerdo con Chenglie Hu (2023), diseñar software es complejo y muchos autores tienen opiniones diferentes, lo que provoca que pocos se atrevan a siquiera dar una definición. No obstante, el autor destaca que la mayoría de los profesionales coinciden en que un buen diseño debe satisfacer los requisitos del software y proporcionar flexibilidad estructural, movilidad y extensibilidad.

Si se piensa en el diseño desde la perspectiva de una disciplina creativa, este puede considerarse como un ejercicio de pensamiento sistemático con sus propios métodos tecnológicos de comunicación, construcción, planificación estratégica e integración sistemática (Knuth, 1974). Una vez que se plantea una perspectiva clara sobre el proceso de diseñado, no se puede dejar de lado el hecho de que modelar sistemas sigue siendo complejo, puesto que, según Hu (2023), no existen soluciones correctas o incorrectas, simplemente existen soluciones que son buenas, no tan buenas o sencillamente malas.

El término de modelado se utiliza ampliamente en diversos contextos, como en la ciencia, economía, ingeniería, etc., con el fin de proporcionar abstracciones de un sistema con cierto nivel de precisión y detalle. La *Object Management Group* (OMG) describe que el modelado se refiere al diseño de aplicaciones de software antes de la codificación; en sintonía con esto, Gomaa (2011) expresa que el modelado de software se utiliza como una parte esencial del proceso de desarrollo, y que, sin embargo, dichos modelos se deben construir y analizar antes de la implementación del sistema, para posteriormente utilizarse en las etapas subsecuentes.

Así mismo, la tarea de elaborar modelos en la etapa de diseño de software se relaciona con la creación de artefactos y de otros productos que sirven para plasmar el modelo en cuestión. De acuerdo con Sommerville (2016), un artefacto es cualquier producto de trabajo tangible que se produce durante el proceso de Ingeniería de Software, el cual sirve como una abstracción del sistema para comunicar decisiones estructurales y analizar el comportamiento del software antes de su implementación.

Un aspecto importante del modelado en términos de diseño de software recae en los problemas de seguridad, ya que, según Rehman & Mustafa (2009), las vulnerabilidades pueden ocurrir en cualquier fase del ciclo de vida del desarrollo de software (SDLC, por sus

siglas en inglés), pero la fase de diseño es la más susceptible a esas vulnerabilidades. Estos mismos autores hacen énfasis en que pequeñas decisiones u omisiones en la fase de diseño pueden ocasionar vulnerabilidades significativas en la aplicación, que los atacantes pueden explotar para comprometer la seguridad del software. Por ejemplo, Hoglund & McGraw (2004) señalan que, durante el fortalecimiento de seguridad de Microsoft en 2002, aproximadamente el 50% de las vulnerabilidades detectadas tuvieron su origen en deficiencias a nivel de diseño.

En este contexto, existe la necesidad implementar medidas de seguridad robustas para proteger sistemas de software de los ciberatacantes —que son cada vez más complejos— en la actualidad. Tras ver la creciente tendencia de la seguridad en el ámbito del software, surge la necesidad de llevar a cabo un diseño seguro, el cual, según la *Open Web Application Security Project* (OWASP, 2025), se refiere a una cultura y a una metodología que se encarga de evaluar constantemente las amenazas, y garantizar que el código esté diseñado y probado de forma robusta para prevenir métodos de ataques conocidos. Por otro lado, la Guía del Cuerpo de Conocimientos de Ingeniería de Software (SWEBOK, 2024), menciona que el diseño para la seguridad busca prevenir el acceso o modificación no autorizados de la información ante ataques, garantizando la continuidad del servicio y la recuperación del sistema; así mismo, sostiene que este tipo de diseño gestiona el comportamiento del software para evitar daños materiales, ambientales o la pérdida de vidas humanas.

Por otra parte, cuando se habla de modelar soluciones para que un sistema de software sea seguro, es importante tomar en cuenta que existen diversos tipos de amenazas para las cuales existe una o varias maneras de poder mitigarlas. Es en este punto donde cobra relevancia el concepto de modelo de amenazas, en el cual Bau & Mitchell (2011) transmiten que se refiere a la especificación explícita de las capacidades, recursos computacionales y niveles de acceso al sistema que posee un atacante potencial, con el propósito de analizar si el diseño del software satisface las propiedades de seguridad establecidas bajo dichas condiciones.

Por lo tanto, si bien existen diversas maneras de diseñar software seguro mediante enfoques, metodologías y técnicas reportadas en la literatura, la falta de un material de referencia sobre artefactos de diseño orientados a la seguridad ocasiona que los practicantes puedan omitir la representación explícita de un modelado seguro o incluso, dar mayor

prioridad a otros atributos de calidad, dejando a la seguridad de manera implícita en el diseño de software.

2. Planteamiento del problema

A medida que la sociedad comenzó a depender más del software, la seguridad se ha vuelto de vital importancia debido a que tiene un impacto real en la vida cotidiana de las personas (Rehman & Mustafa, 2009). Estos mismos autores resaltan que no se puede considerar seguro un software que contenga vulnerabilidades que podrían ser explotadas por atacantes. Así mismo, los autores también hacen énfasis en que uno de los principales problemas del software seguro es la falta de conocimiento sobre seguridad entre los desarrolladores, puesto que, a pesar de que un desarrollador conozca las vulnerabilidades existentes, suele desconocer sus causas y cómo prevenirlas.

Bajo esta misma línea de pensamiento, el hecho de que los profesionales elaboren modelos de diseño sin atender aspectos de seguridad no resulta ajeno a la realidad, ya que, de acuerdo con Ebad (2022), miles de ingenieros de software cada vez tienen menos experiencia en seguridad, debido a que ese aspecto no forma parte de su currículo universitario. Al respecto, el autor profundiza señalando que se han propuesto diferentes técnicas de seguridad para su aplicación en cada fase del ciclo de vida del desarrollo de software (SDLC), abarcando los requisitos, el diseño, la codificación y las pruebas. Sin embargo, Ebad concluye con que el 32% de la investigación en la literatura existente se centra en técnicas de la etapa de diseño. En este mismo sentido, la literatura evidencia que las vulnerabilidades de diseño son la principal fuente de riesgo de seguridad en los sistemas de software, y el 50% de los errores se identifican a menudo en la etapa de diseño (Khan et al., 2021).

No obstante, a pesar de que en la literatura exista información sobre el modelado seguro mediante el uso de artefactos de diseño, el proceso de documentación para los practicantes resulta en una tarea posiblemente compleja, dado que se debe afrontar la labor de búsqueda de información necesaria para el entendimiento y toma de decisiones sobre cómo usar determinado artefacto. Esa información suele estar repartida en diversos lugares, como en artículos de la literatura blanca, así como también en libros o estándares de la

industria pertenecientes a la literatura gris, lo que puede generar un problema de acceso a la información. Por ende, el no tener un recurso de información sobre modelado de seguro mediante el uso de artefactos de diseño de software, puede ocasionar que al practicante de Ingeniería de Software se le dificulte la toma de decisiones sobre qué hacer en situaciones particulares de diseño.

Por lo tanto, existe un problema de toma de decisiones debido al volumen de posibilidades que afronta un practicante de Ingeniería de Software durante el proceso de modelado, en donde debe determinar cuál decisión puede ser la más adecuada en determinado contexto. De este modo, es probable que en la literatura existan muchos artefactos, los cuales pueden ser desconocidos y que, dado la gran cantidad de ellos, probablemente sea complejo tomar decisiones sobre estos artefactos.

3. Objetivos

3.1. Objetivo general

Generar un material de referencia sobre artefactos de diseño orientados a la seguridad clasificados a partir de un modelado de amenazas, a través de una revisión sistemática de la literatura, con la finalidad de brindar un recurso sistematizado y de consulta estructurada que facilite la toma de decisiones durante el proceso de modelado seguro con respecto a la elección y el uso de los artefactos de diseño identificados en la literatura.

3.2. Objetivos específicos

- OE-1: Identificar artefactos de diseño orientados a la seguridad reportados en la literatura dentro del modelado de software seguro.
- OE-2: Analizar los contextos metodológicos o estándares donde se aplican los artefactos identificados.
- OE-3: Relacionar los artefactos identificados con las amenazas que buscan mitigar, mediante un modelado de amenazas.
- OE-4: Analizar los dominios de aplicación o dominios de software donde se aplican los artefactos identificados.

- OE-5: Identificar los retos asociados a la creación o aplicación de los artefactos identificados en la literatura.
- OE-6: Realizar una investigación complementaria posterior a la RSL sobre los artefactos identificados, analizando sus fuentes oficiales o documentación técnica.
- OE-7: Diseñar la estructura del material de referencia sobre artefactos de diseño orientados a la seguridad, definiendo la plantilla o formato de presentación de cada artefacto.
- OE-8: Construir el material de referencia a partir de los resultados de la RSL y de la investigación complementaria, ilustrando ejemplos de aplicación de carácter teórico y conceptual de los artefactos identificados.
- OE-9: Realizar una evaluación del material de referencia.

4. Justificación

La idea de analizar, implementar y mantener los requisitos de seguridad de los sistemas de software requiere planificar dicha disciplina desde etapas tempranas y asegurar continuamente que se mantenga a lo largo del ciclo de vida del desarrollo de software, incluso después de su despliegue, cuando el software evoluciona (Mirakhorli et al., 2020).

Por otro lado, de acuerdo con Sachitano et al. (2004), el creciente problema de que la seguridad del software se aborde demasiado tarde en el proceso de desarrollo, genera como resultado que la seguridad de los sistemas de software sea deficiente y que pueda conducir a otros compromisos de seguridad, como el robo de servicios o información, los cuales ocasionan mayores costos en el mantenimiento y muchos otros costos indirectos como la pérdida de la productividad.

Bau & Mitchell (2011) sostienen que el uso de un marco conceptual uniforme que defina con precisión la seguridad del sistema ayudará a analizar la seguridad, apoyar la evaluación comparativa y desarrollar un conocimiento útil sobre las decisiones de diseño. Bajo esta premisa, se establece que la identificación de artefactos de diseño orientados a la seguridad contribuye a los aspectos del modelado de software seguro, desde su concepción hasta las fases posteriores del ciclo de vida de desarrollo. En este sentido, la identificación

de dichos artefactos no solo implica una extracción de la literatura, sino también un análisis metodológico de los enfoques utilizados para la construcción de estos artefactos. Complementariamente, el estudio contempla la clasificación de estos artefactos a partir de un modelo de amenazas, lo cual permitirá identificar los posibles contextos de uso o situaciones en las que su aplicación resulte adecuada.

En consecuencia, se prevé aportar un recurso sobre el modelado de software seguro en el diseño, el cual pueda servir de referencia en diversos contextos, tomando en cuenta las consideraciones o retos inherentes que puedan existir con la utilidad de cada artefacto. Por lo tanto, la ilustración de ejemplos de aplicación de carácter teórico y conceptual proporcionará un material de valor el cual pretende ser accesible y sistematizado, que sirva de referencia para la toma de decisiones del practicante de Ingeniería de Software.

En este sentido, dirigir la investigación hacia la extracción de artefactos reportados en la literatura, permitirá analizar los procesos utilizados para su construcción, comprender las amenazas más recurrentes y determinar la capacidad preventiva de cada componente. Este esfuerzo se orienta al beneficio de los practicantes de Ingeniería de Software, abarcando desde estudiantes en proceso de formación hasta profesionales en el área.

5. Supuesto

El desarrollo de un material de referencia sobre artefactos de diseño orientados a la seguridad facilitará la toma de decisiones de los practicantes de Ingeniería de Software durante el proceso de modelado seguro en contextos específicos.

6. Alcances y limitaciones

En esta sección se establece la delimitación del trabajo, describiendo tanto los alcances como las limitaciones. El alcance define hasta dónde llegará la investigación, mientras que las limitaciones establecen restricciones o aspectos que la investigación no llega a cubrir por diversos factores.

6.1. Alcances

1. El trabajo abarcará sólo los artefactos identificados en la Revisión Sistemática de la Literatura.
2. La clasificación de los artefactos se hará bajo un modelado de amenazas existente y no se propondrá ninguno propio.
3. Esta propuesta no pretende ser una guía, ya que el material de referencia sólo abarcará situaciones de diseño en las cuales ya hay un contexto establecido, sin contemplar la totalidad de los aspectos del modelado ni el proceso completo del diseño de software.
4. El material de referencia sobre artefactos de diseño incluirá ejemplos de aplicación de carácter teórico y conceptual, sin llegar a validaciones empíricas.
5. El trabajo no incluirá el estudio de herramientas para la creación de los artefactos ni ningún tipo de software para el diseño.

6.2. Limitaciones

1. La investigación de la literatura blanca estará limitada por el acceso a los artículos permitido por la Universidad Veracruzana.
2. La investigación complementaria basada en la literatura gris estará limitada a los recursos económicos, en casos donde se requiera algún tipo de pago para tener acceso a la documentación oficial o documentación especializada de los artefactos.

7. Marco teórico

7.1. Ingeniería de Software

La Ingeniería de Software, según Sommerville et al. (2011) es una disciplina de la ingeniería que se interesa por todos los aspectos relacionados con la producción de software, desde las primeras etapas de la especificación del sistema hasta el mantenimiento del sistema después de que se pone en operación. Sommerville et al., también expresan que la Ingeniería de Software se considera un enfoque sistemático para la producción de

software que toma en cuenta los temas prácticos de costo, fecha y confiabilidad, así como las necesidades de clientes y fabricantes de software.

7.2. Diseño de software

Por otro lado, según la Guía del Cuerpo de Conocimientos de Ingeniería de Software (SWEBOK, 2024) el diseño de software es la aplicación de la disciplina de la Ingeniería de Software en la que se analizan los requisitos del sistema para definir sus características externas y su estructura interna como base para su construcción. La Guía también menciona que el diseño de software se desarrolla en tres etapas: el diseño arquitectónico del sistema, el diseño de alto nivel o externo del sistema y sus componentes y finalmente el diseño detallado o interno.

7.2.1. Modelo y artefacto

Independientemente de las fases, el diseño de software abarca aspectos relacionados con el modelado, término el cual Sommerville et al. (2011) describen como el proceso para desarrollar modelos abstractos de un sistema, donde cada modelo presenta una visión o perspectiva diferente de dicho sistema. De manera general, el modelado se ha convertido en un medio para representar el sistema usando algún tipo de notación gráfica, que casi siempre suele basarse en notaciones de Lenguaje de Modelado Unificado (UML, por sus siglas en inglés). Para elaborar una representación más clara del sistema durante el proceso de modelado, se hace uso de artefactos. Un artefacto es un resultado de trabajo autocontenido que posee un propósito dentro de un contexto específico y que está compuesto por tres niveles: contenido semántico, estructura sintáctica y una representación física (Méndez Fernández et al., 2019).

7.2.2. Atributo de calidad

Es probable que existan diferentes enfoques que se puedan implementar durante el modelado de sistemas de software, en donde se aborden diversos atributos de calidad que el sistema debe satisfacer. Un atributo de calidad (QA, por sus siglas en inglés) es una propiedad medible o comprobable de un sistema que se utiliza para indicar qué tan bien el sistema satisface las necesidades de las partes interesadas, más allá de su función básica

(Len Bass et al., 2021). En términos más sencillos, los autores expresan que se puede pensar en un atributo de calidad como en la medición de la utilidad de un producto a lo largo de alguna dimensión de interés para una parte interesada.

7.2.2.1. Seguridad como atributo de calidad

Los atributos de calidad pueden satisfacer necesidades como una alta disponibilidad, un buen rendimiento o brindar seguridad para proteger el sistema de algún ataque. A pesar de que cada atributo de calidad tiene su relevancia, la seguridad sigue siendo uno de los aspectos más críticos de la calidad del software y el hecho de incorporar la seguridad en cualquier nivel del ciclo de vida del desarrollo de software (SDLC, por sus siglas en inglés) se ha convertido en un requisito urgente (Khan et al., 2022). De esta manera, se entiende la seguridad como una medida de la capacidad que posee un sistema para proteger los datos e información contra el acceso no autorizado, mientras proporciona acceso a personas y sistemas que sí están autorizados (Len Bass et al., 2021).

7.3. Ciberseguridad y desarrollo seguro

La ciberseguridad es el conjunto de prácticas, tecnologías y políticas destinadas a proteger los sistemas digitales, la información y las infraestructuras conectadas frente a amenazas maliciosas, con el fin de preservar valores fundamentales como la privacidad, la confianza y el correcto funcionamiento de la sociedad digital (Herrmann & Pridöhl, 2020).

Así mismo, dada importancia de la seguridad en el desarrollo, Khan et al. (2022) describen al desarrollo seguro como el proceso de diseñar, construir y mantener software incorporando prácticas de seguridad en todas las fases del ciclo de vida del desarrollo, con el objetivo de identificar y mitigar riesgos, vulnerabilidades y amenazas, garantizando las propiedades de confidencialidad, integridad y disponibilidad del sistema.

7.3.1. Diseño de software seguro

Como parte del desarrollo seguro, se tiene que el diseño de software seguro se centra en cómo prevenir la divulgación, creación, modificación, eliminación o denegación de accesos no autorizados a la información y otros recursos, ante ataques al sistema o violaciones de las políticas para limitar daños, proporcionar continuidad del servicio y facilitar la

reparación y la recuperación. Así mismo, también gestiona el comportamiento del software en circunstancias que podrían provocar daños o pérdidas de vidas humanas, daños a la propiedad o al medio ambiente. (SWEBOK, 2024).

7.3.2. Modelado de amenazas

Cuando se habla de seguridad en el desarrollo de software, es posible que la literatura hable sobre amenazas, ataques o vulnerabilidades que un sistema de software posee. Sin embargo, una manera de identificar las amenazas y definir su prevención o mitigación es mediante el uso de un modelado de amenazas. Según la OWASP (2025), se refiere a un proceso estructurado utilizado para identificar, analizar y comprender las amenazas que pueden afectar a un sistema o aplicación, con el propósito de evaluar riesgos de seguridad y definir contramedidas que permitan prevenir o mitigar dichos riesgos.

7.3.3. Ataque

Un ataque en el contexto de la ciberseguridad se define como una acción ofensiva contra la ciberinfraestructura de un objetivo, incluyendo ordenadores, software, redes, procedimientos y personas conectadas (Derbyshire et al., 2018).

7.3.4. Vulnerabilidad

Herrmann & Pridöhl (2020) definen a una vulnerabilidad como una debilidad en la lógica computacional presente en el software y algunos componentes de hardware que, al ser explotados, afectan negativamente la confidencialidad, la integridad o la disponibilidad.

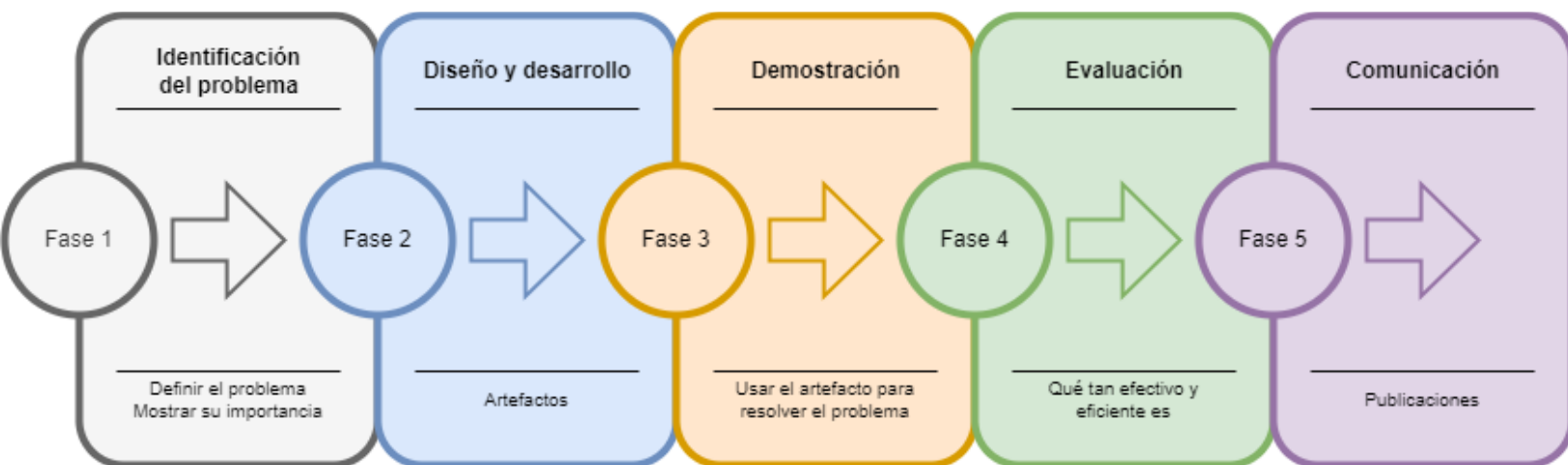
7.3.5. Amenaza

Derbyshire et al. (2018), sostienen que las amenazas pueden definirse como un peligro para la ciberinfraestructura del objetivo. La amenaza presentada es un adversario, conocido como agente de amenaza o actor de amenaza, que buscará identificar y explotar una vulnerabilidad en el sistema objetivo para lograr el impacto deseado.

8. Método

El presente trabajo se desarrolla bajo el marco de la *Design Science Research Methodology* (DSRM) propuesta por Peffers et al. (2008), un enfoque riguroso y capaz de adaptarse a la Ingeniería de Software para la creación y evaluación de productos de trabajo. A diferencia de los estudios limitados a la investigación descriptiva, la DSRM permite estructurar la investigación aplicada orientada a la resolución de problemas mediante el diseño de una solución concreta: en este caso, un material de referencia sobre artefactos de diseño orientados a la seguridad.

Para cumplir con los objetivos de esta investigación, el proceso metodológico se adaptó a las seis actividades secuenciales establecidas por Peffers et al. (2008):



Fase 1. Identificación del problema y motivación

El punto de partida consiste en definir el problema de investigación y justificar el valor de su solución. En esta etapa, se ejecuta una Revisión Sistemática de la Literatura (RSL) basada en la metodología de Kitchenham et al. (2015). Esta técnica rigurosa permite analizar el estado del arte e identificar la diversidad de artefactos de diseño seguro, sus contextos de uso y las amenazas que abordan. Al mismo tiempo, la RSL fundamenta el problema principal: evidenciar las limitaciones, la alta complejidad y los retos operativos que enfrentan los ingenieros al intentar seleccionar e implementar estos artefactos en la práctica.

Fase 2. Definición de los objetivos de una solución

Con base en los hallazgos y retos identificados en la RSL, se infieren los objetivos del artefacto propuesto. El objetivo principal de la solución es proveer un material de referencia estructurado que consolide el conocimiento disperso en la literatura, facilitando a los practicantes de Ingeniería de Software la consulta, comprensión y toma de decisiones sobre qué artefactos utilizar según su contexto tecnológico y modelo de amenazas.

Fase 3. Diseño y desarrollo

Esta actividad abarca la creación del artefacto (el material de referencia). Su desarrollo se divide en dos procesos clave:

- I. **Investigación complementaria:** Se profundiza exclusivamente en los artefactos identificados en la RSL, utilizando documentación técnica, estándares y fuentes oficiales para sustentar su estructura y propósito.
- II. **Construcción del producto:** Se define una arquitectura y una plantilla estandarizada para organizar el contenido. Posteriormente, se integran los datos recopilados para estructurar formalmente el material de referencia.

Fase 4. Demostración

Consiste en demostrar que el artefacto desarrollado resuelve instancias del problema planteado. Para ello, el material de referencia incluirá ejemplos de aplicación de carácter teórico y conceptual. Esta demostración ilustrará cómo la estructura y el contenido del material asisten prácticamente en la selección y comprensión de los modelos de seguridad aplicados a escenarios de diseño específicos.

Fase 5. Evaluación

En esta etapa se observa y mide qué tan bien el material de referencia apoya la solución del problema. Se llevará a cabo una evaluación conceptual del producto final orientada a verificar su coherencia, claridad estructural y utilidad. Dicha

validación se ejecutará mediante la obtención de retroalimentación a través de juicios de expertos o revisiones técnicas, permitiendo analizar las observaciones e iterar los ajustes necesarios para consolidar la versión final del material.

Fase 6. Comunicación

La fase final consiste en documentar y comunicar el problema, la relevancia de la solución, el diseño del artefacto y su utilidad a la comunidad académica y profesional. Esta actividad se materializa mediante la redacción, defensa y publicación del presente trabajo de investigación, garantizando la trazabilidad y la transferencia del conocimiento estructurado.

9. Contenido del trabajo de investigación

A continuación, se presenta la estructura del trabajo de investigación.

1. Capítulo I. Introducción
 - 1.1. Antecedentes
 - 1.2. Planteamiento del problema
 - 1.3. Objetivos
 - 1.3.1. Objetivo general
 - 1.3.2. Objetivos específicos
 - 1.4. Justificación
 - 1.5. Supuesto
 - 1.6. Alcances y limitaciones
 - 1.6.1. Alcances
 - 1.6.2. Limitaciones
2. Capítulo II. Marco teórico
 - 2.1. Ingeniería de software
 - 2.2. Diseño de software
 - 2.2.1. Modelo y artefacto
 - 2.2.2. Atributo de calidad

- 2.2.2.1. Seguridad como atributo de calidad
- 2.3. Ciberseguridad y desarrollo seguro
 - 2.3.1. Diseño de software seguro
 - 2.3.2. Modelado de amenazas
 - 2.3.3. Ataque
 - 2.3.4. Vulnerabilidad
 - 2.3.5. Amenaza
- 3. Capítulo III. Método de investigación
 - 3.1. Revisión Sistemática de la Literatura
 - 3.2. Investigación complementaria de los artefactos
 - 3.3. Diseño del material de referencia
 - 3.4. Construcción del material de referencia
 - 3.5. Evaluación del material de referencia
- 4. Capítulo IV. Análisis
- 5. Capítulo V. Diseño
- 6. Capítulo VI. Implementación
- 7. Capítulo VII. Evaluación
- 8. Capítulo VIII. Conclusiones y trabajo futuro
 - 8.1. Conclusión
 - 8.2. Trabajo futuro
- 9. Referencias
- 10. Apéndices

10. Cronograma de actividades

En la siguiente tabla se presenta el cronograma de actividades detallado, considerando un periodo de 18 meses, tomando en cuenta la elaboración de la RSL y del Trabajo Recepcional.

Fase	Actividades	Meses																	
		2026												2027					
		Feb	Mar	Abr	May	Jun	Jul	Ago	Sep	Oct	Nov	Dic	Ene	Feb	Mar	Abr	May	Jun	Jul
Protocolo de investigación	Redactar los antecedentes																		
	Redactar el planteamiento del problema																		
	Redactar los objetivos																		
	Redactar la justificación																		
	Redactar el supuesto																		
	Redactar los alcances y limitaciones																		
	Redactar el marco teórico																		
	Redactar el método																		
	Redactar el contenido del trabajo																		
	Elaborar el cronograma																		
	Redactar las preguntas de investigación																		
Revisión Sistemática de la Literatura	Establecer los objetivos de la RSL																		
	Identificar las fuentes de información																		
	Elaborar las cadenas de búsqueda																		
	Establecer los criterios de inclusión y exclusión																		
	Realizar las búsquedas																		
	Establecer la precisión de las búsquedas																		
	Aplicar el filtrado																		
	Elaborar una matriz de extracción																		
Investigación complementaria de los artefactos	Redactar el informe de la RSL																		
	Identificar fuentes técnicas y estándares																		
	Localizar documentación oficial de los artefactos																		
	Extraer especificaciones de estructura y uso de cada artefacto																		
	Documentar retos técnicos y consideraciones de aplicación																		
	Sintetizar la información técnica para la descripción detallada																		
Diseño del material de referencia	Consolidar los insumos técnicos finales																		
	Seleccionar el modelo de amenazas base para la organización																		
	Definir categorías y taxonomía del catálogo																		
	Determinar los campos de información obligatorios																		
	Diseñar la plantilla estandarizada del producto																		
	Establecer criterios de presentación visual y formato																		
Construcción del material de referencia	Validar la coherencia interna de la estructura propuesta																		
	Redactar las fichas técnicas por cada artefacto																		
	Diseñar ejemplos teóricos y conceptuales para cada ficha																		
	Integrar hallazgos de la RSL y documentación técnica																		
	Generar ilustraciones y posibles diagramas de aplicación																		
Evaluación del material de referencia	Ensamblar la versión preliminar del catálogo																		
	Definir el instrumento de evaluación (cuestionario/juicio)																		
	Seleccionar expertos o practicantes para la revisión																		
	Aplicar la evaluación de utilidad y claridad																		
	Analizar resultados y retroalimentación obtenida																		
	Realizar ajustes finales al contenido y estructura																		
	Generar la versión final del material de referencia																		

Tabla I.I. Cronograma de actividades.

Referencias

- Bau, J., & Mitchell, J. C. (2011). Security modeling and analysis. *IEEE Security and Privacy*, 9(3), 18–25. <https://doi.org/10.1109/MSP.2011.2>
- Derbyshire, R., Green, B., Prince, D., Mauthe, A., & Hutchison, D. (2018). An Analysis of Cyber Security Attack Taxonomies. *Proceedings - 3rd IEEE European Symposium on Security and Privacy Workshops, EURO S and PW 2018*, 153–161. <https://doi.org/10.1109/EuroSPW.2018.00028>
- Ebad, S. A. (2022a). Exploring How to Apply Secure Software Design Principles. *IEEE Access*, 10, 128983–128993. <https://doi.org/10.1109/ACCESS.2022.3227434>
- Ebad, S. A. (2022b). What topics should or should not be included in software security education—Qualitative content analysis. *Computer Applications in Engineering Education*, 30(6), 1753–1773. <https://doi.org/10.1002/cae.22554>
- Gomaa, H. (2011). *Software modeling and design: UML, use cases, patterns, and software architectures*. <https://books.google.com/books?hl=es&lr=&id=TqZi-hAX17YC&oi=fnd&pg=PR7&dq=what+is+software+modeling&ots=4XXbG74uJL&sig=KwG006FsTnQDtFpboa0kyiaCsV0>
- Herrmann, D., & Pridöhl, H. (2020). Basic Concepts and Models of Cybersecurity. *International Library of Ethics, Law and Technology*, 21, 11–44. https://doi.org/10.1007/978-3-030-29053-5_2
- Hoglund, G., & McGraw, G. (2004). *Exploiting Software: How to Break Code*. <http://www.cigital.com>
- Hu, C. (2023). An Introduction to Software Design: Concepts, Principles, Methodologies, and Techniques. *An Introduction to Software Design: Concepts, Principles, Methodologies, and Techniques*, 1–359. <https://doi.org/10.1007/978-3-031-28311-6>
- Khan, R. A., Khan, S. U., Khan, H. U., & Ilyas, M. (2021). Systematic Mapping Study on Security Approaches in Secure Software Engineering. *IEEE Access*, 9, 19139–19160. <https://doi.org/10.1109/ACCESS.2021.3052311>
- Khan, R. A., Khan, S. U., Khan, H. U., & Ilyas, M. (2022). Systematic Literature Review on Security Risks and its Practices in Secure Software Development. *IEEE Access*, 10, 5456–5481. <https://doi.org/10.1109/ACCESS.2022.3140181>
- Knuth, D. E. (1974). Computer Programming as an Art. *Communications of the ACM*, 17(12), 667–673. <https://doi.org/10.1145/361604.361612>

- Len Bass, Paul Clements, & Rick Kazman. (2021). *Software Architecture in Practice Fourth Edition*.
- Méndez Fernández, D., Böhm, W., Vogelsang, A., Mund, J., Broy, M., Kuhrmann, M., & Weyer, T. (2019). Artefacts in software engineering: a fundamental positioning. *Software & Systems Modeling* 18:5, 18(5), 2777–2786.
<https://doi.org/10.1007/s10270-019-00714-3>
- Mirakhorli, M., Galster, M., & Williams, L. (2020). Understanding Software Security from Design to Deployment. *ACM SIGSOFT Software Engineering Notes*, 45(2), 25–26.
<https://doi.org/10.1145/3385678.3385687>
- OWASP. (2025a). *A06 Diseño inseguro - OWASP Top 10:2025*.
https://owasp.org/Top10/2025/A06_2025-Insecure_Design/
- OWASP. (2025b). *Thread Modeling*. https://owasp.org/www-community/Threat_Modeling
- Rehman, S., & Mustafa, K. (2009). Research on software design level security vulnerabilities. *ACM SIGSOFT Software Engineering Notes*, 34(6), 1–5.
<https://doi.org/10.1145/1640162.1640171>
- Sachitano, A., Chapman, R. O., & Hamilton, J. A. (2004). Security in software architecture: A case study. *Proceedings From the Fifth Annual IEEE System, Man and Cybernetics Information Assurance Workshop, SMC*, 370–376.
<https://doi.org/10.1109/iaw.2004.1437841>
- Sommerville. (2016). *Software Engineering* (10th ed.).
- Sommerville, I., Columbus, B., New, I., San, Y., Upper, F., River, S., Cape, A., Dubai, T., Madrid, L., Munich, M., Montreal, P., Delhi, T., São, M. C., Sydney, P., Kong, H., Singapore, S., & Tokyo, T. (2011). *SOFTWARE ENGINEERING Ninth Edition*.
- Washizaki, H. (Ed.). (2024). *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide) Version 4.0* (4th ed.). IEEE Computer Society.

“Lis de Veracruz: Arte, Ciencia, Luz”

www.uv.mx

